

Mandate — Claude Code Build Plan (ETHOnline 2026)

Team: You (UI/UX lead, full-time, owns the demo video) + your friend (backend/chain lead)

Window: Fri Sept 4, 2026 → Sun Sept 13, 2026, 12:00pm EDT (**9:30pm IST**) — ~9.5 days, not

12 **Targets:** The Graph, Arc/Circle, ENS as the 3 official Partner Prize picks; 1inch Aqua as a stretch goal

0. How to use this document

You're both running Claude Code, in the same repo, on two parallel tracks:

- **Track B (Backend/Chain)** — (Siddharth) Identity, data composition, settlement, execution engine.
- **Track U (UI/UX)** — (Nisarg). Design system, dashboards, the "blocked transaction" demo moment, and everything a judge actually looks at.

Every phase below is tagged with its track. Some phases run in parallel (same calendar days, different tracks); some are integration points where both of you sync.

Before either of you writes a single line of project code:

1. Both of you open Claude Code in the shared repo.
 2. Paste the **Master Context Prompt** (Section 2) into *both* sessions, verbatim. This is what stops the two sessions from silently disagreeing about architecture, naming, or chain choices three days in.
 3. Read Section 3 (the Loop Protocol) once, together. Every phase prompt after this assumes you both know what "run the loop" means — I won't re-explain it 14 times.
 4. Then, and only then, start Phase 1 on Sept 4.
-

1. Read this before Sept 4 — compliance landmines

"Start Fresh" is stricter than it sounds. Per the rules: *"any prior project-specific code, designs, or assets are not allowed."* That includes Figma mockups, wireframes, a color palette you fell in love with, or a half-written contract — not just code. Between now (Sept 2) and Sept 4, the only things you're allowed to do without risking your partner-prize/Finalist eligibility are:

reading docs, creating dev accounts (Subgraph Studio, Circle developer portal, 1inch developer portal), getting testnet ETH, and setting up an *empty* repo with generic tooling (ESLint config, a bare Next.js scaffold from the public starter templates the sponsors link — not anything Mandate-specific). Phase 0 below is scoped exactly to that boundary. Don't let the design itch jump the gun.

Commit like a human engineer, document like an honest one. No per-commit AI signatures needed (that's not an ETHGlobal requirement). Small, honestly-described, incremental commits — never one giant end-of-day dump (the rules explicitly call this out as disqualification-risk). Section 4 has the exact format. Section 5 has the AI_USAGE.md spec that carries the actual required disclosure.

Live data only, for Graph and 1inch. Both sponsors explicitly disqualify mocked/local-only data. Every phase touching Graph or Aqua below says "live" for a reason — don't let Claude Code quietly reach for a fixture file under time pressure.

2. MASTER CONTEXT PROMPT

(Paste this into both Claude Code sessions before Phase 1. It's long on purpose — this is what the whole build stands on.)

We are building "Mandate" for ETHOnline 2026 (ETHGlobal), a fully online hackathon.

Submission deadline: Sunday Sept 13 2026, 12:00pm EDT. No late submissions accepted.

WHAT WE'RE BUILDING, IN PLAIN LANGUAGE

Mandate lets someone deploy an autonomous DeFi trading agent that cannot exceed its own declared authority, and that earns or loses trust based on its real track record — enforced on-chain, not by trusting the agent's own good behavior.

Concretely, four pieces wired into one pipeline:

1. IDENTITY & PERMISSIONS: each agent gets an ENSv2 subname (e.g. alpha.fund.eth) on Sepolia.

Its Enhanced Access Control record IS its permission scope — which protocols, which position types, max position size, max daily spend. Public, checkable, not a config file.

2. REPUTATION & RISK SIGNAL: before acting, we compose two Graph products — the sponsor-listed

Agent0/ERC-8004 Subgraph (the agent's real job history / validated outcomes) plus a subgraph

we build ourselves indexing Mandate's own past decisions and fills — into one live

trust-and-risk score. This is served over a public MCP server for natural-language queries.

3. ENFORCEMENT & EXECUTION: a custom SwapVM opcode, MandateGate, added to 1inch's Aqua/SwapVM

opcode table, reads the agent's permission scope and reputation score AT EXECUTION TIME and

reverts the fill if either check fails. This is enforced inside the swap itself, not as an

off-chain pre-check an attacker or a bug could route around.

4. SETTLEMENT & REPUTATION WRITE-BACK: approved trades fund and settle in USDC over Arc's

(Circle's) Agent Stack. The outcome gets written back into the agent's reputation record,

closing the loop for its next decision.

WHY THIS IS REAL, NOT A TOY PROBLEM (cite this in the README, it's true and sourced):

2026 has seen repeated, costly failures of exactly this kind — a prompt-injected agent wallet

lost ~\$150K twice, another agent lost track of its own balance after a crash and sent 1000x

its intended transfer (~\$250K gone), and MCP memory-poisoning attacks drained an estimated

\$45M from a cluster of Solana trading-agent platforms. ERC-8004 (identity/reputation) and

tools like AgentScope (on-chain spend limits via ERC-4337) are the ecosystem's current answer

to HALF the problem — enforcement. Nobody has wired live reputation + market risk into the

enforcement decision itself. That's the gap we're filling. Say this explicitly in the README —

name ERC-8004 and AgentScope directly. Don't let a judge think we've never heard of them.

TARGET SPONSOR TRACKS (submit for up to 3 official Partner Prizes; build the 4th as a stretch):

- The Graph: "Best Use of Composable/Standardized Graph Products" AND "Best AI Tooling/Use

Case" (both are reachable from one integration — see WHAT WE'RE BUILDING #2)

- Arc/Circle: "Best Agentic Economy with Circle Agent Stack" AND "Best DeFi Stablecoin-Native Pool" (both reachable from #4)

- ENS: "Best Use of ENSv2" (from #1)

- 1inch (STRETCH, attempt only after the above three work end to end): "Build an Aqua App" (from #3)

ARCHITECTURE DECISION — READ CAREFULLY, THIS IS NOT OBVIOUS:

ENSv2 Enhanced Access Control only exists on Sepolia (beta). 1inch's Aqua/SwapVM contracts

are NOT guaranteed to be deployed on Sepolia — verify their actual deployment status in

Phase 1 before assuming a chain. This means the permission record (Sepolia) and the

MandateGate opcode (wherever Aqua lives) are very likely on DIFFERENT chains, and a smart

contract cannot synchronously read state on a different chain.

The correct fix, not a workaround: deploy a small "PermissionMirror" contract on the SAME

chain as our Aqua/SwapVM deployment. A backend relay service watches the canonical ENSv2

Sepolia record and the composed Graph reputation score, and keeps PermissionMirror in sync

whenever either changes. MandateGate reads PermissionMirror synchronously, on its own chain,

at execution time — so enforcement is still a same-chain, unbypassable on-chain read, and the relay is just a sync mechanism, not a trust boundary. Confirm and document this design in ARCHITECTURE.md in Phase 1 before other phases build on top of it.

TEAM & DIVISION OF LABOR

- Backend/Chain (Track B): identity/permission contracts, Graph subgraph + composition logic, Arc settlement, the MandateGate opcode, the MCP server.
- UI/UX (Track U): design system, the underwriting dashboard (visible policy strip / kill switch / trade log — this is a judged UX pattern, not decoration), the "blocked transaction" demo moment, treasury view, final demo video.

GLOBAL ENGINEERING STANDARDS (apply to every phase, both tracks)

- Real, live data only where a sponsor's rules require it (Graph, 1inch both explicitly disqualify mocked/local-only data). Testnet is fine; fixtures/mocks standing in for the real integration are not.
- Small, honest, incremental commits. See the Git Discipline section.
- Every phase ends with an update to AI_USAGE.md (see Section 5) and a pass through the Loop Protocol (see Section 3) — don't skip either under time pressure.
- If you (Claude) are not certain an API, contract address, or function signature is real, say so explicitly and verify before using it. Guessing a plausible-looking signature is the single most likely way this project breaks two days before the deadline.
- Prefer TypeScript across the stack (frontend, backend, subgraph mappings) for one consistent language, unless a sponsor's tooling forces otherwise (e.g. Substreams in Rust).

Confirm you've understood this by summarizing back: what Mandate is, the 4 pieces, the chain/architecture decision, and which track you are (Backend or UI). Then wait for the Phase 1 prompt.

3. The Loop Protocol (anti-hallucination, run this on every task)

This is the single most important section for keeping Claude Code honest across 9 days of momentum and fatigue. Paste this once per session near the start of each phase, or just say "run this task under the Loop Protocol" once Claude Code has seen it.

LOOP PROTOCOL — follow this for every task in this phase, no exceptions:

1. RESTATE the task in your own words. List every file you expect to touch and every external thing (contract, API, SDK function, subgraph field) you expect to depend on.
2. VERIFY BEFORE YOU BUILD. For every external dependency listed above: find it for real — in the actual installed package source (node_modules), the official docs we've linked, or the sponsor's GitHub repo — and quote the exact signature, address, or field name you found. If you cannot find it, or you're extrapolating from a similar-sounding thing, STOP and write "UNVERIFIED: <the specific thing>" instead of guessing. Ask me rather than inventing a plausible default. This is the rule that matters most — a confidently invented contract address or SDK method is worse than an honest "I don't know yet."
3. IMPLEMENT the smallest coherent unit of work — not the whole task at once.
4. SELF-CHECK. Actually run the build, linter, type-checker, and any relevant tests. Paste the real output. If something fails, diagnose the actual error message (not a guess at what might be wrong), form one specific hypothesis, apply one fix, and re-run. Repeat up to 3 times. If still broken after 3 tries, STOP — report exactly what's broken, what you

tried, and what you think the real blocker is. Do not paper over a failure or claim

something "should work now" without having seen it work.

5. REPORT back: what changed, what you actually verified (with evidence), what's still a stub or an assumption (log it in ASSUMPTIONS.md, one line per item), and what — if anything — you need from me before continuing.

6. WAIT for a go-ahead before starting the next unit, unless I've explicitly told you to run the rest of this phase autonomously.

7. COMMIT only working, verified code, in the format from Section 4. Never bundle unrelated changes into one commit because you're behind schedule.

A task is only "done" if you ran it and watched it work. "This should work" is not done.

4. Git & commit discipline

COMMIT RULES for this project:

- One logical change per commit. Not "day 3 progress", not "fixes".
- Imperative mood, present tense, like a real engineer: "Add ENSv2 subname resolver for agent permission lookup", not "Added stuff for ENS" and not "🤖 AI-generated ENS module".
- No AI-tool signatures or emoji tags in the commit message itself — that's not required by ETHGlobal's rules (the disclosure requirement is satisfied by AI_USAGE.md and the README, see Section 5), and it's not standard git practice.
- Commit as you go, multiple times a day, not once at the end of a session. The rules explicitly flag "large single commits or missing histories" as a disqualification risk —

judges are checking for a real, visible progression, not just a clean final state.

- Never commit a secret, private key, or API key. .env in .gitignore from commit #1.
 - Before each commit: append the corresponding entry to AI_USAGE.md (Section 5).
-

5. AI_USAGE.md — the actual attribution requirement

Create this file in Phase 0 and append to it after every work session on both tracks. This is what satisfies ETHGlobal's rule, and per the Spec-Driven Development clause, a thorough one is a *plus*, not a liability.

AI Usage

Mandate was built using Claude Code (Anthropic) as a development assistant across both the backend/chain and UI/UX tracks. This file documents where and how, updated per work session.

Format (one entry per session)

- Date, track (Backend/UI), phase
- What Claude Code was asked to do
- What was AI-generated vs. human-directed/reviewed/modified
- Any spec files or planning prompts used (linked, and included in /specs)

Entries

(appended chronologically by both of you as you work — see Section 3 step 7)

Also create a `/specs` folder in the repo now and drop this exact document into it (`/specs/build-plan.md`) plus every phase prompt you actually paste into Claude Code, verbatim, as you go. The rules ask for this explicitly: *"Judges need to see the full picture of how you directed the AI, not just the generated output."* This folder is free credibility — don't skip it.

6. Phase-by-phase plan

Phase 0 — Pre-event setup only (Sept 2–3, before official start)

Track: Both. **Boundary:** research and accounts only — no Mandate-specific code, contracts, or designs (see Section 1).

- Both: create accounts — Subgraph Studio, 1inch Developer Portal (API key), Circle Developer Portal (Arc testnet access), get Sepolia + Arc testnet + Base Sepolia test funds from their respective faucets.
- Backend track: read the actual current state of 1inch's github.com/1inch/aqua and github.com/1inch/swap-vm repos — confirm whether a testnet deployment exists yet, or whether you'll be deploying the official contracts yourselves (this was true for several teams as recently as mid-2026 — don't assume it's changed).
- Backend track: read ENSv2 Enhanced Access Control docs (docs.ens.domains/ensv2/enhanced-access-control) and the Agent0/ERC-8004 Subgraph docs (thegraph.com/docs/en/subgraphs/existing-subgraphs/agent0/).
- UI track: research only — look at atlas, Aqua Prime, and AgentArena's actual showcase pages (ethglobal.com/showcase) for how they visualized policy/permission state. Do not start Figma files yet.
- Both: scaffold an *empty* repo — generic Next.js + TypeScript starter, ESLint/Prettier config, [.gitignore](#), [.env.example](#). No Mandate-specific files.
- Deliverable: repo exists and builds; accounts + testnet funds ready; [ARCHITECTURE.md](#) and [AI_USAGE.md](#) created as empty templates.

Phase 1 — Identity core: ERC-8004 + ENSv2 permission schema (Sept 4)

Track: Backend (lead), UI starts Phase 2 in parallel same day.

PHASE 1 PROMPT (Backend/Chain session)

Run this under the Loop Protocol.

Goal: stand up the agent identity layer.

1. Confirm and write ARCHITECTURE.md with: (a) which chain hosts our Aqua/SwapVM deployment

(verify via the actual repos, don't assume), (b) the PermissionMirror design from the

Master Context Prompt, (c) exact contract addresses/chain IDs we'll use for Sepolia (ENS), the execution chain (Aqua), and Arc testnet.

2. Register a test agent identity via ERC-8004's Identity Registry. Find the real registry contract address for our target testnet by checking the EIP-8004 reference implementation and Agent0 docs — do not invent an address. If no registry is deployed where we need it, document the plan to deploy our own instance and do so.
 3. Register an ENSv2 subname on Sepolia (e.g. testagent.mandate.eth) and define the Enhanced Access Control schema for a permission record: allowed protocols (list), allowed position types (enum), max position size (uint), max daily spend (uint), expiry. Get one subname reading AND writing this record end to end — read it back and print it, don't just submit the write transaction.
 4. Store a human-readable policy summary in the ENS text records too (precedent: Aqua Prime stored its agent's identity/personality this way) — e.g. a `mandate.policy` text record with a plain-English summary of the same permissions.
 5. Write the PermissionMirror contract skeleton (empty logic is fine for now) on the execution chain, and a backend relayer script stub that will sync it from the ENS record.
- Definition of done: a script that registers an ERC-8004 identity, writes an ENSv2 permission record + text-record policy summary for it, and reads both back, printing real on-chain data (not console.log of local variables) — run against live Sepolia testnet.
-

Phase 2 — UI/UX foundation: design system + core screens (Sept 4–6, parallel to Phase 1/3)

Track: UI (lead).

PHASE 2 PROMPT (UI/UX session)

Run this under the Loop Protocol.

Goal: build the design system and wireframe the core screens BEFORE live data exists, using realistic placeholder data clearly marked as such (never claim placeholder data is real).

1. Design system: define a small token set (colors, type scale, spacing) — this is a fintech/infra product, not a consumer app; look at how atlas and Aqua Prime presented policy/trust state for reference (already researched in Phase 0), don't default to generic AI-page aesthetics (no purple gradient hero, no default shadow look with zero customization).
2. Core screens to wireframe then build as real components (React + our chosen styling approach — confirm and lock this in before building more than one screen):
 - a. Agent overview: identity (ENS name), current permission scope, current trust score.
 - b. Underwriting dashboard: THIS IS THE JUDGED PATTERN — a visible policy strip (what this agent can/can't do right now), a kill switch (visibly revoke authority), and a live trade log (every decision, approved or blocked, with the reason). Model this on what atlas built, not a generic table.
 - c. The "blocked transaction" screen — an attempted action that fails the MandateGate check, shown clearly with WHY it failed. This is your best demo-video moment; design it to be legible at a glance, not buried in a log.

- d. Treasury/settlement view (for Phase 6 to wire up later).
3. Build these with realistic example data (mark it clearly as example data in code comments) so Backend can swap in real data sources later without a redesign.

Definition of done: all four screens exist as real components, responsive, in both a light and dark treatment, reviewed against real component states (loading, empty, error — not just the happy path), committed with example data clearly marked as such.

Phase 3 — Composed Graph data layer + underwriting logic (Sept 5–7)

Track: Backend (lead).

PHASE 3 PROMPT (Backend/Chain session)

Run this under the Loop Protocol.

Goal: the actual trust-and-risk composition.

1. Query the real, live Agent0/ERC-8004 Subgraph for our registered test agent's reputation data via Subgraph Studio (live, not mocked — this is a hard requirement for the Graph track). Confirm the actual schema fields by reading the subgraph's GraphQL schema, don't guess field names.
2. Build our OWN subgraph indexing Mandate's own on-chain events (permission writes, gate checks, fills, settlements) — deploy it to Subgraph Studio, confirm it's actually indexing live testnet events by triggering one and watching it appear in a query.
3. Write the composition/underwriting logic: given (a) the agent's ENS permission scope, (b) its ERC-8004 reputation, (c) our own indexed history, compute a single trust-and-risk score and an allow/deny decision for a proposed action. This must be real logic with a

documented scoring formula in code comments — not a stub that always returns true.

4. Stand up a public MCP server exposing this in natural language (e.g. "what can alpha.fund.eth do right now, and what's its track record?") — this satisfies the Graph AI track. Confirm it actually responds to a real query, end to end, before calling this done.

Definition of done: querying the MCP server about a real registered test agent returns a real, composed answer sourced from live Subgraph Studio data — demonstrate this with an actual transcript in the phase report, not a description of what it would do.

Phase 4 — Underwriting dashboard wired to real data (Sept 6–8)

Track: UI (lead), depends on Phase 1 (identity) and Phase 3 (composed data) interfaces being stable enough to consume.

PHASE 4 PROMPT (UI/UX session)

Run this under the Loop Protocol.

Goal: replace every piece of example data in Phase 2's screens with real data from Backend's Phase 1/3 outputs.

1. Sync with Backend on the actual API/query shape now available (their MCP server / direct subgraph queries / contract reads) — do not assume the shape from Phase 2's placeholders, confirm it against what Phase 3 actually shipped.
2. Wire the agent overview and underwriting dashboard screens to real data: real ENS permission scope, real composed trust score, real trade log entries from our own subgraph.
3. Handle real-world states properly: what does the dashboard show while a Graph query is still indexing, if the subgraph has zero events yet, if an ENS record read fails? Build

and show all three, don't just handle the happy path.

4. The kill switch must actually call the real permission-revocation transaction, not just update local UI state — confirm it changes the on-chain record and the dashboard reflects that change after a real re-fetch.

Definition of done: open the dashboard cold, with no cached state, against live testnet data, and see accurate real information — screen-record this moment, you'll likely reuse the clip.

Phase 5 — Arc/Circle settlement + reputation write-back (Sept 7–9)

Track: Backend (lead).

PHASE 5 PROMPT (Backend/Chain session)

Run this under the Loop Protocol.

Goal: real USDC movement tied to real decisions, and the reputation loop closing.

1. Integrate Circle's Agent Stack (github.com/circlefin/agent-stack-starter-kits — read the actual starter kit code before assuming its API shape) to give our test agent a Circle wallet on Arc testnet.
2. Build the funding/settlement flow: an approved action (one that passed the Phase 3 underwriting check) triggers a real USDC transfer on Arc testnet — confirm the tx hash is real and visible on Arc's block explorer, don't just log a simulated success.
3. Tie the decision logic explicitly to the composed signal from Phase 3 (Arc's own rules ask for "agents with clear decision logic tied to real signals" — make this traceable: the settlement record should reference which trust score and which permission check it

passed).

4. Write the outcome back to the agent's reputation record (via our own subgraph from Phase 3, and to ERC-8004's Reputation Registry if it's write-accessible to us — verify this, don't assume write access exists without checking).
5. Run one full loop manually: propose action → underwriting check passes → Arc settlement executes → reputation updates → re-querying the MCP server shows the updated history.

Definition of done: one complete, real, live-testnet cycle, with every step's on-chain evidence (tx hashes, explorer links) captured in the phase report.

Phase 6 — Treasury/settlement UI (Sept 8–9, parallel to Phase 5's back half)

Track: UI (lead).

PHASE 6 PROMPT (UI/UX session)

Run this under the Loop Protocol.

Goal: make the money movement from Phase 5 visible and legible.

1. Build the treasury view: current Circle/Arc wallet balance (real, fetched live — not cached or hardcoded), a settlement history list linking each entry back to the underwriting decision that authorized it (reuse the trade-log pattern from Phase 2/4, keep the visual language consistent).
2. Every settlement entry should be clickable through to its real Arc testnet explorer transaction — confirm these links actually resolve to the real transaction, not a guessed URL pattern.

3. Add a simple funding flow (deposit USDC into the agent's Circle wallet) if Backend's

Phase 5 exposes one — confirm what's actually available before designing around an assumed feature.

Definition of done: treasury view shows real, live Arc testnet balance and settlement history, every entry traceable to a real transaction and a real underwriting decision.

Phase 7 — MandateGate: the custom SwapVM opcode (Sept 9–11) — STRETCH GOAL

Track: Backend (lead). Only start after Phases 1, 3, and 5 are solid end to end — this is the highest-risk piece, timeboxed deliberately.

PHASE 7 PROMPT (Backend/Chain session)

Run this under the Loop Protocol. This phase has a hard timebox: if it isn't working by Sept 11 evening, fall back per the instruction at the end — don't let it eat the docs/video buffer.

Goal: enforce the underwriting decision inside the swap itself, not before it.

1. Confirm the real, current SwapVM opcode table structure by reading the actual github.com/1inch/swap-vm source — do not assume an opcode format from memory or from a similar project's description.
2. Deploy the PermissionMirror contract (skeleton from Phase 1) for real on the execution chain, and get the Phase 5 relay logic actually syncing it from the canonical ENS record + composed trust score.
3. Write a new opcode, MandateGate, following the precedent pattern of Turing Swap's `_humanGate` (reads an external state source mid-execution and reverts on failure) —

appended to the opcode table, not replacing existing ones, per 1inch's rules ("bonus points for modifying opcodes", official contracts required as the base).

4. Wire it into one real Aqua position type. Demonstrate BOTH outcomes live on testnet: an in-scope, well-reputed action executing successfully, AND an out-of-scope or low-reputation action reverting at the MandateGate check — with the actual on-chain revert visible, not simulated.
5. Proper git history for this phase specifically — 1inch's rules explicitly flag "no single-commit entries on the final day" as disqualifying for this track.

FALLBACK if this isn't cleanly working by Sept 11 evening: execute the approved action as a plain swap instead of a custom SwapVM position, keep the underwriting story intact via Phases 1–5, and don't submit for the 1inch partner prize. The core three tracks don't depend on this phase.

Definition of done: two real, live testnet transactions — one MandateGate-approved fill that succeeds, one MandateGate-blocked attempt that reverts on-chain — both with tx hashes.

Phase 8 — Execution UI: the demo-moment screen (Sept 9–11, parallel to Phase 7)

Track: UI (lead).

PHASE 8 PROMPT (UI/UX session)

Run this under the Loop Protocol.

Goal: this is the single most important screen in your demo video. Build it to be understood in under 5 seconds by someone who's never seen the project before.

1. Build the trade-attempt flow: propose an action, watch it move through the underwriting check in real time, see it either execute (with a real Arc settlement) or get blocked (with the real MandateGate revert reason) — sourced from Backend's Phase 7 output once available; build against Phase 5's off-chain check first so you're not blocked waiting.
2. Design the "blocked" state to be unmistakable and satisfying to watch — this is your wow-factor moment per the judging criteria. Show the specific rule that was violated in plain language, not a raw revert string.
3. Confirm this screen works against BOTH real outcomes from Phase 7's definition of done before considering it finished.

Definition of done: a stranger who's never seen Mandate can watch this screen for 15 seconds and correctly explain what just happened, unprompted.

Phase 9 — Integration pass + bug bash (Sept 11)

Track: Both, together, live.

PHASE 9 PROMPT (both sessions, working together)

Run this under the Loop Protocol, but the goal now is verification, not new features.

1. Walk the full user journey end to end, together, on live testnets: register agent → set permissions → propose actions (one approved, one blocked) → settlement → reputation update → dashboard reflects all of it correctly.
2. For every step, actually click through it as a first-time user would — don't verify via code review alone.

3. Log every rough edge in ASSUMPTIONS.md / a new ISSUES.md, triaged as "fix before submission" vs "known limitation, mention honestly in README."
4. Do NOT start new feature work today unless the walkthrough reveals a hard blocker for one of the 3 core tracks' qualification requirements.

Definition of done: the full happy path and the full blocked-path both work, live, start to finish, with no manual data seeding via a script that wouldn't exist in production.

Phase 10 — Documentation: README, ARCHITECTURE, FEEDBACK, AI_USAGE (Sept 11–12)

Track: Both — you lead this since you own the overall submission, Backend fills in the technical sections.

PHASE 10 PROMPT (either session, coordinate the split)

Run this under the Loop Protocol for factual accuracy — do not let this phase invent capabilities the project doesn't actually have.

1. README.md: what Mandate is (from the Master Context Prompt, tightened), the architecture diagram (from ARCHITECTURE.md), setup instructions a judge can actually follow to run it, and — explicitly — a short section naming ERC-8004 and AgentScope as prior art and stating what Mandate adds on top. Every claim in this file must be checked against what actually works, not what was planned.
2. Finish AI_USAGE.md (Section 5) — this should already be mostly populated from ongoing entries; do a final pass for completeness, not a first draft.
3. Confirm /specs contains this build plan and every phase prompt actually used.
4. Per-sponsor submission notes: for each of Graph, Arc, ENS (and 1inch if it shipped),

write the specific paragraph the submission form asks for — how we used/integrated their tools, honestly, referencing real qualification requirements from their prize page.

5. Uniswap-style FEEDBACK.md is NOT required for our three core tracks — skip unless you end up using Uniswap stack somewhere and want that bounty too.

Definition of done: a judge with zero prior context can read the README, understand what Mandate is and why it's real, and successfully run it locally or against our live testnet deployment.

Phase 11 — Demo video (Sept 12)

Track: You, solo (per your role).

Not a Claude Code phase — but Claude Code can help you script it. Use this prompt in either session:

PHASE 11 PROMPT (scripting help)

Help me write a 2:30–3:30 minute demo video script for Mandate, following ETHGlobal's rules:

- Under 20 seconds of backstory/intro
- Show the project actually running — no more than a few seconds of "waiting", edit around it
- Lead with the blocked-transaction demo moment from Phase 8, since it's the most immediately legible "wow" beat
- No more than 4 bullet points per any summary slide
- Real spoken narration only — no AI voiceover, no text-over-music
- Structure: what problem (10s) → what Mandate does, shown live (2min) → the 3 sponsor integrations, named explicitly and shown, not just claimed (45s) → close

Base the script only on what we actually built and verified in Phases 1-10 — don't script a capability we didn't ship.

Recording checklist (from the rules, non-negotiable):

- 720p minimum, real screen recording, not a phone camera
- 2–4 minutes hard limits, don't speed it up to fit
- Record with a decent mic in a quiet room; do at least 2 takes

Phase 12 — Submission (Sept 12 evening – Sept 13, before 9:30pm IST)

Track: You.

1. Final read-through of README + AI_USAGE.md + /specs against what's actually in the repo.
2. Confirm real git history exists across the whole window (spot-check: no single mega-commit).
3. Fill out the Hacker Dashboard submission form.
4. Select up to 3 Partner Prizes — default to **Graph, Arc, ENS**; swap in 1inch only if Phase 7 fully succeeded and you're confident dropping one of the other three is worth it.
5. Upload the demo video; confirm it processes successfully (resolution/length checks happen on upload — don't submit at the last minute in case a re-upload is needed).
6. Submit with buffer before 9:30pm IST on Sept 13 — not at the deadline.

7. Quick reference: what "done" looks like per sponsor

Sponsor	Non-negotiable qualification detail
The Graph	Live Subgraph Studio data, not mocked; 2+ Graph products composed; reasoning shown, not raw query output
Arc	Working frontend + backend + an architecture diagram; decision logic traceably tied to a real signal

Sponsor	Non-negotiable qualification detail
ENS	ENSv2 (Sepolia) central to the product, functional demo, no hard-coded values
1inch (stretch)	Official Aqua/SwapVM contracts; real Git history; on-chain token transfer shown in the demo

Good luck — paste the Master Context Prompt and go.